

# NEXTSTEP

Security Operations Platform

## Backend Deployment Plan

### Multi-Tenant SOC-Driven DMARC Analysis & Response Platform

Version 1.0 | March 2026 | Classification: Confidential

| Field       | Details                                                          |
|-------------|------------------------------------------------------------------|
| Project     | DMARC Analyzer Platform — PFE03-2026-SOC                         |
| Scope       | Backend Architecture, API Services, Workers, Data Layer & DevOps |
| Authors     | Backend Engineering — NEXTSTEP                                   |
| Status      | Ready for Implementation                                         |
| Target Env. | Docker / Docker Compose (Dev) → Kubernetes (Production)          |

# 1. Executive Overview

---

This document defines the complete backend deployment plan for the DMARC Analyzer Platform — a multi-tenant, SOC-driven system for ingesting, parsing, analyzing, and responding to DMARC aggregate (RUA) and forensic (RUF) reports. It is derived from the functional requirements defined in PFE03-2026-SOC and the UML design architecture document (v0.1).

The backend is the operational core of the platform, responsible for:

- Automated DMARC report collection via IMAP
- XML parsing, normalization and structured storage
- SPF / DKIM / DMARC alignment analysis and anomaly detection
- IP enrichment, risk scoring and threat intelligence integration
- Real-time alerting, SIEM/SOAR push and IOC generation
- REST + WebSocket API exposure for the frontend and third-party integrations
- Full multi-tenant RBAC governance

The frontend is considered delivered. This plan covers only the backend, its infrastructure, data layer, and DevOps pipeline.

## 2. Technology Stack — Full Specification

### 2.1 Core Runtime & Framework

| Layer          | Technology                        | Version               | Justification                                                     |
|----------------|-----------------------------------|-----------------------|-------------------------------------------------------------------|
| Runtime        | Python                            | 3.12                  | Ecosystem dominance for security tooling, async support, typing   |
| API Framework  | FastAPI                           | 0.111+                | Async-native, OpenAPI auto-generation, Pydantic v2 validation     |
| Task Queue     | Celery                            | 5.4+                  | Distributed workers, beat scheduler for IMAP polling, retry logic |
| Message Broker | Redis                             | 7.2 (via Redis Stack) | Celery broker + result backend + pub/sub for real-time events     |
| WebSocket      | FastAPI WebSocket + Redis Pub/Sub | —                     | Real-time dashboard updates and alert streaming                   |
| ASGI Server    | Uvicorn + Gunicorn                | 0.30+                 | Production-grade process management with async workers            |

### 2.2 Data Layer

| Store              | Technology               | Version     | Role                                                    |
|--------------------|--------------------------|-------------|---------------------------------------------------------|
| Primary DB         | PostgreSQL               | 16          | Tenants, RBAC, Users, Config, Policies, Recommendations |
| ORM                | SQLAlchemy               | 2.0 (async) | Async queries, migrations, relationship modeling        |
| Migrations         | Alembic                  | 1.13+       | Schema versioning, rollback-safe migrations             |
| Search / Analytics | Elasticsearch            | 8.13        | High-volume DMARC record indexing and SOC querying      |
| ES Client          | elasticsearch-py (async) | 8.13+       | Async bulk indexing, aggregation queries                |
| Cache              | Redis                    | 7.2         | API response cache, session store, rate limiting        |
| Object Storage     | MinIO (S3-compatible)    | Latest      | Raw report XML/ZIP archival — immutable forensic store  |

### 2.3 Security & Authentication

| Component    | Technology                   | Notes                                                   |
|--------------|------------------------------|---------------------------------------------------------|
| Auth         | JWT (RS256)                  | Short-lived access tokens (15min) + refresh tokens (7d) |
| OAuth2 / SSO | python-jose + FastAPI OAuth2 | Supports SSO integration for enterprise tenants         |

| Component         | Technology                          | Notes                                                         |
|-------------------|-------------------------------------|---------------------------------------------------------------|
| RBAC              | Custom middleware                   | Roles: NEXTSTEP Admin, Client Admin, SOC Analyst, Client User |
| Tenant Isolation  | Row-Level Security (PostgreSQL RLS) | Every query scoped to tenant_id via RLS policies              |
| Password Hashing  | bcrypt (passlib)                    | Cost factor 12                                                |
| TLS               | Let's Encrypt / Cert-Manager (K8s)  | TLS termination at Nginx/Traefik ingress                      |
| Secrets           | HashiCorp Vault / Docker Secrets    | No secrets in env files or images                             |
| API Rate Limiting | slowapi + Redis                     | Per-tenant and per-endpoint limits                            |

## 2.4 External Integrations & Enrichment

| Service         | Library / Method               | Purpose                                      |
|-----------------|--------------------------------|----------------------------------------------|
| IMAP Collection | imaplib / aioimaplib           | Poll dedicated mailboxes for DMARC reports   |
| DNS Lookup      | dnspython (async)              | SPF/DKIM/DMARC record validation             |
| GeoIP           | MaxMind GeoLite2 (mmdb-reader) | Country + city resolution from IP            |
| ASN             | pyasn + BGP data               | Autonomous System Number lookup              |
| Threat Intel    | AbuseIPDB API / VirusTotal API | IP reputation scoring                        |
| Email Alerts    | aioSMTPlib / SendGrid API      | Alert notifications to SOC analysts          |
| Slack/Teams     | httpx (webhook)                | ChatOps integration for critical alerts      |
| SIEM Push       | REST/Syslog (httpx)            | CEF/JSON IOC export to SIEM/SOAR             |
| Ticketing       | Jira/ServiceNow REST API       | Automated ticket creation on critical alerts |

## 2.5 DevOps & Infrastructure

| Tool               | Technology                       | Role                                          |
|--------------------|----------------------------------|-----------------------------------------------|
| Containerization   | Docker 26 + Docker Compose v2    | Service isolation, local dev stack            |
| Orchestration      | Kubernetes (K8s) 1.30            | Production HA deployment, auto-scaling        |
| Ingress            | Nginx Ingress Controller         | TLS termination, routing, rate limiting       |
| CI/CD              | GitHub Actions                   | Lint → Test → Build → Push → Deploy pipeline  |
| Container Registry | GitHub Container Registry (GHCR) | Private image storage                         |
| Monitoring         | Prometheus + Grafana             | Metrics, dashboards, alerts                   |
| Log Aggregation    | Loki + Grafana                   | Centralized log shipping from all containers  |
| Tracing            | OpenTelemetry + Jaeger           | Distributed trace across services             |
| Health Checks      | FastAPI /health + K8s probes     | Liveness and readiness probes on all services |



## 3. Backend Architecture

### 3.1 Service Decomposition

The backend is organized into discrete, independently deployable services, each with a single responsibility. This maps directly to the component architecture defined in the UML document (Figure 17 & 18).

| Service               | Type          | Responsibility                                                               |
|-----------------------|---------------|------------------------------------------------------------------------------|
| api-gateway           | FastAPI App   | All REST endpoints, WebSocket, JWT auth, RBAC enforcement, rate limiting     |
| collector-worker      | Celery Worker | IMAP polling, attachment download, decompression, raw storage to MinIO       |
| parser-worker         | Celery Worker | XML schema validation, DMARC parsing, normalization, DB + ES indexing        |
| analysis-worker       | Celery Worker | SPF/DKIM/DMARC alignment checks, conformance scoring, anomaly flagging       |
| correlation-worker    | Celery Worker | Cross-event correlation, spoofing pattern detection, incident classification |
| scoring-worker        | Celery Worker | Risk score computation, threat intel enrichment, GeoIP/ASN resolution        |
| recommendation-worker | Celery Worker | Policy maturity evaluation, SPF/DKIM fix generation, DMARC progression       |
| alert-service         | Celery Worker | Alert creation, severity routing, channel notification (email/Slack/SIEM)    |
| scheduler             | Celery Beat   | Cron-based triggers: IMAP polling, re-scoring, cleanup tasks                 |
| flower                | Flower UI     | Celery task monitoring and worker health dashboard                           |

### 3.2 Request Flow — API

Client (Frontend / 3rd Party) → Nginx Ingress (TLS) → api-gateway (FastAPI) → PostgreSQL / Elasticsearch / Redis → Response

### 3.3 Async Processing Pipeline

The pipeline follows an event-driven chain. Each step publishes to a Celery queue consumed by the next worker:

| Step        | Queue         | Worker           | Output                                         |
|-------------|---------------|------------------|------------------------------------------------|
| 1. Schedule | beat          | scheduler        | Triggers collect task per mailbox config       |
| 2. Collect  | collect.queue | collector-worker | Raw XML/ZIP stored in MinIO, DB record created |

| Step         | Queue           | Worker                | Output                                                    |
|--------------|-----------------|-----------------------|-----------------------------------------------------------|
| 3. Parse     | parse.queue     | parser-worker         | Structured DmarcReport + ReportRecord rows in PostgreSQL  |
| 4. Analyse   | analysis.queue  | analysis-worker       | SPF/DKIM/DMARC results, alignment flags, conformance rate |
| 5. Correlate | correlate.queue | correlation-worker    | Incident classification, spoofing pattern tags            |
| 6. Score     | score.queue     | scoring-worker        | Risk score per Source entity, threat intel enrichment     |
| 7. Recommend | recommend.queue | recommendation-worker | Recommendation records persisted, policy stage updated    |
| 8. Alert     | alert.queue     | alert-service         | Alert created, SOC notified, SIEM push if configured      |

## 4. Database Design

### 4.1 PostgreSQL Schema — Core Entities

All tables include `created_at`, `updated_at` timestamps and are scoped with `tenant_id` for multi-tenancy. Row-Level Security (RLS) is enforced at the PostgreSQL level.

#### 4.1.1 Tenant & Identity

| Table       | Key Columns                                            | Notes                                                           |
|-------------|--------------------------------------------------------|-----------------------------------------------------------------|
| tenants     | id, name, slug, plan, is_active                        | Root isolation unit for all data                                |
| users       | id, tenant_id, email, hashed_password, role, is_active | RBAC: nextstep_admin   client_admin   soc_analyst   client_user |
| memberships | id, user_id, tenant_id, role, invited_at               | User-to-tenant many-to-many with role override                  |
| audit_logs  | id, tenant_id, user_id, action, resource, metadata, ts | Immutable event log for all state changes                       |
| api_keys    | id, tenant_id, user_id, key_hash, name, last_used_at   | Third-party API access tokens                                   |

#### 4.1.2 Domain & Mailbox Configuration

| Table           | Key Columns                                                                                                             | Notes                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| domains         | id, tenant_id, fqdn, dmarc_policy, spf_record, dkim_selectors[], status                                                 | Monitored domain registry                 |
| mailbox_configs | id, tenant_id, domain_id, imap_host, imap_port, username, encrypted_password, folder, poll_interval_min, last_polled_at | Encrypted credentials, per-domain mailbox |
| dmarc_policies  | id, domain_id, policy (none quarantine reject), pct, rua_uri, ruf_uri, version                                          | Current policy snapshot                   |
| policy_versions | id, domain_id, policy_id, changed_by, diff, created_at                                                                  | Immutable policy change history           |

#### 4.1.3 Report Ingestion

| Table          | Key Columns                                                                                                        | Notes                                       |
|----------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| report_raws    | id, tenant_id, domain_id, mailbox_config_id, s3_key, filename, content_type, size_bytes, received_at, parse_status | Forensic evidence store — never deleted     |
| dmarc_reports  | id, tenant_id, domain_id, raw_id, reporter_org, report_id, date_range_begin, date_range_end, policy_published      | Parsed aggregate report header              |
| report_records | id, report_id, source_ip, source_id, count, policy_disposition, dkim_aligned, spf_aligned, header_from             | One row per IP/policy combination in report |



| Table         | Key Columns                                                     | Notes                          |
|---------------|-----------------------------------------------------------------|--------------------------------|
| spf_results   | id, record_id, domain, result (pass fail softfail neutral none) | Detailed SPF auth results      |
| dkim_results  | id, record_id, domain, selector, result (pass fail none)        | Detailed DKIM auth results     |
| dmARC_results | id, record_id, result, override_reason                          | Final DMARC disposition result |

#### 4.1.4 Source Intelligence

| Table                | Key Columns                                                                                             | Notes                                  |
|----------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------|
| sources              | id, tenant_id, ip, asn, asn_org, country_code, city, is_known_provider, first_seen, last_seen           | Unique sending source entity           |
| source_risk_scores   | id, source_id, score (0-100), risk_level (low medium high critical), threat_intel_data jsonb, scored_at | Latest score + full enrichment payload |
| source_score_history | id, source_id, score, risk_level, scored_at                                                             | Time-series score evolution            |

#### 4.1.5 Incidents, Alerts & Recommendations

| Table           | Key Columns                                                                                                                                                    | Notes                               |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| incidents       | id, tenant_id, domain_id, title, type, severity, status, source_ids[], evidence jsonb                                                                          | Spoofing incident container         |
| alerts          | id, tenant_id, incident_id, domain_id, severity, status (new acknowledged in_progress escalated resolved false_positive closed), assigned_to, notes, closed_at | SOC alert lifecycle                 |
| alert_events    | id, alert_id, user_id, action, from_status, to_status, comment, created_at                                                                                     | Full state transition audit trail   |
| recommendations | id, tenant_id, domain_id, category (spf dkim dmARC_policy alignment), priority, title, description, suggested_value, current_value, is_resolved, resolved_at   | Auto-generated remediation guidance |

## 4.2 Elasticsearch Index Mapping

All DmarcReport + ReportRecord + Source data is dual-written to Elasticsearch for fast SOC querying. Index strategy:

| Index                     | Shards | Key Mapped Fields                                                                               | Use Case                                       |
|---------------------------|--------|-------------------------------------------------------------------------------------------------|------------------------------------------------|
| dmARC-records-{tenant_id} | 3      | source_ip, domain, date, spf_result, dkim_result, dmARC_result, country, asn, risk_score, count | Primary SOC search, dashboards, trend analysis |
| dmARC-alerts-{tenant_id}  | 1      | severity, status, assigned_to, created_at, domain_id, incident_type                             | Alert queue, escalation filtering              |

| Index                     | Shards | Key Mapped Fields                              | Use Case                               |
|---------------------------|--------|------------------------------------------------|----------------------------------------|
| dmARC-sources-{tenant_id} | 2      | ip, asn, country, risk_level, last_seen, score | Source reputation view, threat hunting |

## 5. REST API Design

### 5.1 API Conventions

- Base URL: /api/v1/
- Authentication: Bearer JWT in Authorization header
- Tenant scope: Resolved from JWT claims (tenant\_id). NEXTSTEP admin can pass ?tenant\_id= override
- Pagination: Cursor-based for large sets, offset for small lists
- Response envelope: { data, meta, errors }
- OpenAPI 3.1 spec auto-generated at /api/docs and /api/redoc

### 5.2 API Route Map

| Module          | Route Prefix     | Key Endpoints                                                                                    |
|-----------------|------------------|--------------------------------------------------------------------------------------------------|
| Auth            | /auth            | POST /login, POST /refresh, POST /logout, POST /register-tenant                                  |
| Tenants         | /tenants         | GET /me, PATCH /me, GET / (admin), POST / (admin), DELETE /{id} (admin)                          |
| Users           | /users           | GET /, POST /, GET /{id}, PATCH /{id}, DELETE /{id}, POST /{id}/roles                            |
| Domains         | /domains         | GET /, POST /, GET /{id}, PATCH /{id}, DELETE /{id}, GET /{id}/policy                            |
| Mailboxes       | /mailboxes       | GET /, POST /, PATCH /{id}, DELETE /{id}, POST /{id}/test, POST /{id}/trigger                    |
| Reports         | /reports         | GET / (paginated, filtered), GET /{id}, GET /{id}/records                                        |
| Records         | /records         | GET / (ES-backed, full filter), GET /{id}, GET /export                                           |
| Sources         | /sources         | GET /, GET /{id}, GET /{id}/history, GET /{id}/records                                           |
| Alerts          | /alerts          | GET /, GET /{id}, PATCH /{id}/status, POST /{id}/assign, POST /{id}/comment, POST /{id}/escalate |
| Incidents       | /incidents       | GET /, GET /{id}, POST /{id}/close                                                               |
| Recommendations | /recommendations | GET /, GET /{id}, PATCH /{id}/resolve                                                            |
| Analytics       | /analytics       | GET /conformance, GET /risk-trend, GET /top-sources, GET /volume, GET /spf-dkim-breakdown        |
| Integrations    | /integrations    | GET /, POST /, PATCH /{id}, DELETE /{id}, POST /{id}/test                                        |
| IOC Feed        | /ioc             | GET /feed (JSON/STIX), GET /feed.csv                                                             |
| WebSocket       | /ws              | ws://host/api/v1/ws/alerts — real-time push for new alerts                                       |

## 6. Worker Modules — Implementation Detail

---

### 6.1 Collector Worker

Triggered by Celery Beat on configurable intervals per mailbox (default: 15 min). Connects via IMAP IDLE or polling. Handles:

- TLS-secured IMAP connection using aioimaplib
- Folder scanning (INBOX + DMARC-specific subfolder)
- Attachment extraction: .zip, .gz, .xml
- Decompression via zipfile + gzip modules
- Raw bytes uploaded to MinIO at path: `/tenant_id/{domain}/{yyyy-mm}/{report_id}.xml`
- ReportRaw record created in PostgreSQL with `parse_status = PENDING`
- parse task published to `parse.queue`
- Duplicate detection via `report_id` hash

### 6.2 Parser Worker

Consumes `parse.queue`. Validates XML against the DMARC RUA schema (draft-ietf-dmarc-aggregate-reporting-03). Extracts:

- Report metadata: `reporter_org`, `report_id`, `date_range_begin/end`, `policy_published`
- Per-record data: source IP, SPF/DKIM results, alignment, disposition, `header_from`, count
- Normalizes provider quirks (Google vs. Microsoft vs. Proofpoint XML variations)
- Bulk inserts `DmarcReport`, `ReportRecord`, `SPFResult`, `DKIMResult`, `DmarcResult`
- Bulk indexes to Elasticsearch `dmarc-records-{tenant_id}` index
- Publishes analysis task to `analysis.queue`
- Sets `parse_status = PARSED` or `FAILED` with error detail

### 6.3 Analysis Worker

Performs per-record and per-report analysis:

- SPF alignment check: `envelope_from` domain vs. from header domain
- DKIM alignment check: `d=` tag vs. from header domain (strict / relaxed per policy)
- DMARC conformance rate calculation per domain per time window
- Non-conformant source flagging: sources with DMARC fail + volume threshold
- Publishes correlation task and scoring task in parallel

### 6.4 Correlation Worker

Implements the detection logic for spoofing attempts:

- Repeated auth failures from same IP across multiple reports: threshold configurable
- Volume anomaly detection: Z-score on rolling 7-day window per source
- New source detection: IPs not previously seen sending for this domain
- Pattern classification: Legitimate / Suspicious / Potentially Malicious
- Incident creation if severity threshold exceeded

- Publishes alert task if incident created

## 6.5 Scoring Worker

Computes a composite 0–100 risk score per Source entity:

- Base score from DMARC failure rate (40% weight)
- Volume anomaly contribution (20% weight)
- Threat intelligence score from AbuseIPDB / VirusTotal (25% weight)
- Geographic and ASN risk factor (15% weight)
- Score thresholds: Low (<40), Medium (40–70), High (71–90), Critical (>90)
- Full enrichment data stored in `source_risk_scores.threat_intel_data` JSONB
- Source state machine transition enforced: New → Observed → Scored → risk band

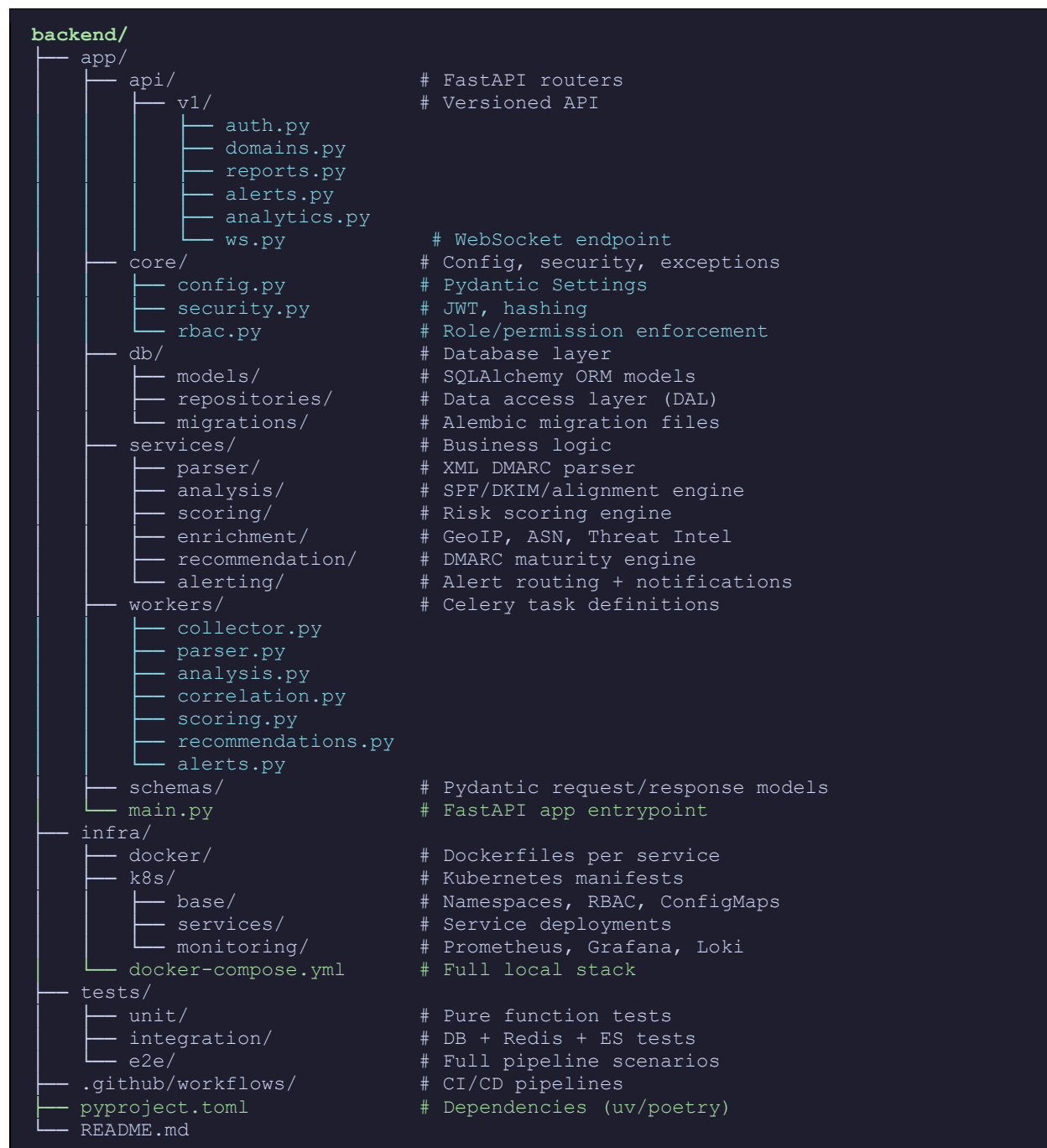
## 6.6 Recommendation Worker

Evaluates the DMARC maturity level of each domain and generates actionable fixes:

- SPF analysis: include mechanisms, too-many-lookups detection (RFC 7208 limit: 10)
- DKIM selector validation: checks alignment and key rotation cadence
- Policy progression logic: `p=none` → `p=quarantine` → `p=reject` with pct ramping
- Auto-generates human-readable Recommendation records with `current_value` vs. `suggested_value`
- Maturity score: 0 (no enforcement) → 100 (full reject, aligned)

## 7. Project Directory Structure

The repository is organized as a monorepo with clear separation between services:



## 8. Docker Compose — Local Development Stack

The following docker-compose.yml brings up the complete backend stack locally for development and testing:

```
version: "3.9"

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: dmarc_db
      POSTGRES_USER: dmarc_user
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dmarc_user"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis/redis-stack:7.2.0-v9
    volumes:
      - redisdata:/data

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.13.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
    volumes:
      - esdata:/usr/share/elasticsearch/data

  minio:
    image: minio/minio:latest
    command: server /data --console-address ':9001'
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
    volumes:
      - miniodata:/data

  api:
    build:
      context: .
      dockerfile: infra/docker/Dockerfile.api
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: postgresql+asyncpg://dmarc_user:${POSTGRES_PASSWORD}@postgres/dmarc_db
      REDIS_URL: redis://redis:6379/0
      ELASTICSEARCH_URL: http://elasticsearch:9200
      MINIO_ENDPOINT: minio:9000
    depends_on: [postgres, redis, elasticsearch, minio]
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

  worker-collector:
    build:
      context: .
      dockerfile: infra/docker/Dockerfile.worker
    command: celery -A app.workers.celery_app worker -Q collect.queue -c 2 -n collector@%h
    environment: &worker-env
      DATABASE_URL: postgresql+asyncpg://dmarc_user:${POSTGRES_PASSWORD}@postgres/dmarc_db
      REDIS_URL: redis://redis:6379/0
```

```
    ELASTICSEARCH_URL: http://elasticsearch:9200
    MINIO_ENDPOINT: minio:9000
  depends_on: [postgres, redis, minio]

worker-parser:
  build: { context: ., dockerfile: infra/docker/Dockerfile.worker }
  command: celery -A app.workers.celery_app worker -Q parse.queue -c 4 -n parser@%h
  environment: *worker-env
  depends_on: [postgres, redis, elasticsearch]

worker-analysis:
  build: { context: ., dockerfile: infra/docker/Dockerfile.worker }
  command: celery -A app.workers.celery_app worker -Q
analysis.queue,correlate.queue,score.queue,recommend.queue>alert.queue -c 4
  environment: *worker-env
  depends_on: [postgres, redis, elasticsearch]

scheduler:
  build: { context: ., dockerfile: infra/docker/Dockerfile.worker }
  command: celery -A app.workers.celery_app beat --loglevel=info
  environment: *worker-env
  depends_on: [redis]

flower:
  image: mher/flower:2.0
  ports: ["5555:5555"]
  command: celery --broker=redis://redis:6379/0 flower
  depends_on: [redis]

volumes:
  pgdata:
  redisdata:
  esdata:
  miniodata:
```



## 9. Environment Variables Reference

All secrets must be injected via Docker Secrets or a Vault-backed secret store. Never commit a .env file with real values.

| Variable                        | Required | Description                                                   |
|---------------------------------|----------|---------------------------------------------------------------|
| DATABASE_URL                    | Yes      | Async PostgreSQL DSN — postgresql+asyncpg://user:pass@host/db |
| REDIS_URL                       | Yes      | Redis connection string — redis://host:6379/0                 |
| ELASTICSEARCH_URL               | Yes      | ES endpoint — http://host:9200                                |
| MINIO_ENDPOINT                  | Yes      | MinIO host:port (no schema)                                   |
| MINIO_ACCESS_KEY                | Yes      | MinIO / S3 access key                                         |
| MINIO_SECRET_KEY                | Yes      | MinIO / S3 secret key — use Vault                             |
| MINIO_BUCKET_REPORTS            | Yes      | Target bucket name for raw DMARC reports                      |
| JWT_PRIVATE_KEY                 | Yes      | RS256 private key PEM — use Vault or K8s Secret               |
| JWT_PUBLIC_KEY                  | Yes      | RS256 public key PEM                                          |
| JWT_ACCESS_TOKEN_EXPIRE_MINUTES | No       | Default: 15                                                   |
| JWT_REFRESH_TOKEN_EXPIRE_DAYS   | No       | Default: 7                                                    |
| ABUSEIPDB_API_KEY               | Prod     | AbuseIPDB threat intelligence API key                         |
| VIRUSTOTAL_API_KEY              | Prod     | VirusTotal API key for IP enrichment                          |
| MAXMIND_DB_PATH                 | Prod     | Path to GeoLite2-City.mmdb file                               |
| SMTP_HOST                       | Yes      | SMTP relay for email alerts                                   |
| SMTP_PORT                       | Yes      | Default: 587                                                  |
| SMTP_USER                       | Yes      | SMTP authentication username                                  |
| SMTP_PASSWORD                   | Yes      | SMTP password — use Vault                                     |
| SLACK_WEBHOOK_URL               | Optional | Slack incoming webhook for alert notifications                |
| SIEM_ENDPOINT_URL               | Optional | Target SIEM/SOAR REST endpoint for IOC push                   |
| LOG_LEVEL                       | No       | Default: INFO                                                 |
| ENVIRONMENT                     | Yes      | development   staging   production                            |
| CORS_ORIGINS                    | Yes      | Comma-separated allowed frontend origins                      |
| ENCRYPTION_KEY                  | Yes      | Fernet key for mailbox credential encryption                  |

## 10. CI/CD Pipeline

### 10.1 Pipeline Stages

| Stage             | Trigger             | Steps                                                            | Gate                     |
|-------------------|---------------------|------------------------------------------------------------------|--------------------------|
| Lint & Format     | Every push          | ruff check, ruff format --check, mypy --strict                   | Fail on error            |
| Security Scan     | Every push          | bandit -r app/, safety check                                     | Fail on HIGH severity    |
| Unit Tests        | Every push          | pytest tests/unit/ --cov=app --cov-report=xml                    | >= 80% coverage          |
| Integration Tests | Every push          | docker-compose -f compose.test.yml up; pytest tests/integration/ | All pass                 |
| Build Images      | main + release tags | docker build per service; tag with SHA + semver                  | Build success            |
| Push to GHCR      | main + release tags | docker push ghcr.io/nextstep/dmarc-*:sha                         | Push success             |
| Deploy Staging    | main branch         | kubectl apply -k infra/k8s/overlays/staging/                     | Smoke tests pass         |
| Deploy Production | Release tag v*.*.*  | kubectl apply -k infra/k8s/overlays/production/                  | Manual approval required |

### 10.2 Dockerfile Pattern (API Service)

```
# Stage 1: deps
FROM python:3.12-slim AS deps
WORKDIR /app
RUN pip install uv
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev

# Stage 2: runtime
FROM python:3.12-slim AS runtime
WORKDIR /app
RUN addgroup --system app && adduser --system --ingroup app app
COPY --from=deps /app/.venv /app/.venv
COPY app/ ./app/
ENV PATH="/app/.venv/bin:$PATH"
USER app
EXPOSE 8000
CMD ["gunicorn", "app.main:app", "-k", "uvicorn.workers.UvicornWorker",
    "-w", "4", "-b", "0.0.0.0:8000"]
```

## 11. Kubernetes Deployment (Production)

### 11.1 Namespace & Resource Strategy

| Namespace        | Contents                                                       |
|------------------|----------------------------------------------------------------|
| dmARC-platform   | All application workloads (API, workers, scheduler)            |
| dmARC-data       | PostgreSQL, Redis, Elasticsearch, MinIO (via Helm or Operator) |
| dmARC-monitoring | Prometheus, Grafana, Loki, Jaeger                              |
| dmARC-ingress    | Nginx Ingress Controller, cert-manager                         |

### 11.2 Deployment Manifests — Key Specs

| Workload         | Kind       | Replicas | Resource Requests    | Resource Limits      | HPA               |
|------------------|------------|----------|----------------------|----------------------|-------------------|
| api              | Deployment | 2–10     | 250m CPU / 256Mi RAM | 1 CPU / 1Gi RAM      | Yes (CPU 70%)     |
| worker-collector | Deployment | 1–3      | 100m CPU / 128Mi RAM | 500m CPU / 512Mi RAM | Yes (queue depth) |
| worker-parser    | Deployment | 2–8      | 250m CPU / 512Mi RAM | 1 CPU / 2Gi RAM      | Yes (queue depth) |
| worker-analysis  | Deployment | 2–6      | 250m CPU / 512Mi RAM | 1 CPU / 2Gi RAM      | Yes (queue depth) |
| scheduler        | Deployment | 1        | 100m CPU / 128Mi RAM | 250m CPU / 256Mi RAM | No (singleton)    |
| flower           | Deployment | 1        | 100m CPU / 128Mi RAM | 250m CPU / 256Mi RAM | No                |

### 11.3 K8s Kustomize Overlays

- base/ — shared resources: Namespace, ServiceAccount, RBAC, ConfigMap, Services
- overlays/staging/ — reduced replicas, debug logging, staging secrets
- overlays/production/ — full replicas, HPA, PodDisruptionBudget, NetworkPolicy

### 11.4 Key K8s Resources

- Ingress: TLS via cert-manager + Let's Encrypt, routing /api/\* → api service
- HorizontalPodAutoscaler: on api + parser + analysis workers based on CPU + custom queue depth metric
- PodDisruptionBudget: minAvailable: 1 on all critical services
- NetworkPolicy: ingress only from ingress controller; workers only reach Redis/PG/ES; no internet egress except enrichment workers
- Secrets: K8s Secrets sealed with Sealed Secrets or Vault Agent Injector
- PersistentVolumeClaim: for MinIO and PostgreSQL in self-hosted deployments



## 12. Observability Stack

### 12.1 Metrics — Prometheus

| Metric                         | Source             | Alert Threshold                        |
|--------------------------------|--------------------|----------------------------------------|
| dmARC_reports_ingested_total   | parser-worker      | Rate drop > 50% vs 1h avg              |
| dmARC_parse_failures_total     | parser-worker      | > 5 failures in 5min                   |
| celery_queue_depth{queue}      | Redis exporter     | Collect queue > 100, parse queue > 500 |
| dmARC_risk_score_high_sources  | scoring-worker     | > 10 new high-risk sources in 1h       |
| dmARC_alerts_created_total     | alert-service      | Spike > 3x baseline                    |
| api_request_duration_seconds   | FastAPI middleware | p99 > 500ms                            |
| api_error_rate                 | FastAPI middleware | > 1% 5xx in 5min                       |
| postgres_connections_active    | postgres-exporter  | > 80% of max_connections               |
| elasticsearch_indexing_latency | ES exporter        | p95 > 1s                               |

### 12.2 Logging — Loki

- All services log to stdout in structured JSON format
- Fields: timestamp, level, service, tenant\_id, request\_id, message, error (if any)
- Promtail DaemonSet ships logs from all pods to Loki
- Grafana dashboards: per-service log explorer, error drill-down, tenant-scoped views
- Retention: 30 days hot, 1 year cold (object storage)

### 12.3 Distributed Tracing — OpenTelemetry + Jaeger

- OTel SDK instrumented in FastAPI middleware and all Celery tasks
- Trace spans: API request → Celery task publish → worker execution → DB writes → ES indexing
- Trace context propagated via task headers across the full pipeline
- Jaeger UI for latency analysis and bottleneck identification

## 13. Security Hardening

### 13.1 Application Security

| Control                  | Implementation                                                                      |
|--------------------------|-------------------------------------------------------------------------------------|
| Multi-tenancy isolation  | PostgreSQL RLS on all tables — tenant_id enforced at DB level, not app level        |
| Credential encryption    | Mailbox IMAP passwords encrypted at rest with Fernet (AES-128-CBC)                  |
| SQL injection prevention | SQLAlchemy parameterized queries only — no raw SQL with user input                  |
| Input validation         | Pydantic v2 strict models on all API endpoints — schema-validated before processing |
| DMARC XML injection      | XML parsed with defusedxml — prevents XXE attacks in DMARC reports                  |
| CORS                     | Strict whitelist via CORS_ORIGINS env variable                                      |
| Rate limiting            | slowapi: 100 req/min per user on standard endpoints, 10/min on auth endpoints       |
| JWT security             | RS256 asymmetric signing — private key never leaves Vault/K8s Secret                |
| API key hashing          | SHA-256 hash stored — raw key only shown once at creation                           |
| Audit logging            | All create/update/delete operations logged to audit_logs with user + IP             |

### 13.2 Infrastructure Security

- All container images built from python:3.12-slim — no root user in containers
- Read-only root filesystem on all K8s pods except write mounts
- Network policies enforce microsegmentation — workers cannot reach internet except enrichment APIs
- Secrets stored in Vault or Sealed Secrets — never in plaintext ConfigMaps
- Container image scanning: Trivy in CI pipeline — fail on CRITICAL CVEs
- Dependency scanning: pip-audit / safety in CI pipeline

## 14. Implementation Roadmap

### Phase 1 — Foundation (Weeks 1–3)

| Task                                                                    | Owner   | Output                |
|-------------------------------------------------------------------------|---------|-----------------------|
| Repository setup, pyproject.toml, uv lockfile, pre-commit hooks         | DevOps  | Repo structure ready  |
| PostgreSQL schema design + Alembic migrations (all tables)              | Backend | DB schema v1 deployed |
| Core FastAPI app: config, RBAC middleware, error handlers, health check | Backend | API gateway skeleton  |
| JWT auth endpoints: login, refresh, logout, register-tenant             | Backend | Auth working          |
| Docker Compose full stack (PG + Redis + ES + MinIO + API)               | DevOps  | Local stack running   |
| GitHub Actions: lint + test pipeline                                    | DevOps  | CI pipeline green     |
| Elasticsearch index templates per entity                                | Backend | ES indices created    |

### Phase 2 — Ingestion Pipeline (Weeks 4–6)

| Task                                                                 | Owner   | Output                    |
|----------------------------------------------------------------------|---------|---------------------------|
| Celery app setup, queue config, beat scheduler                       | Backend | Task infrastructure ready |
| Collector worker: IMAP connection, attachment download, MinIO upload | Backend | Reports collected         |
| Parser worker: defusedxml parsing, normalization, DB + ES dual-write | Backend | Reports parsed            |
| Domain + Mailbox CRUD API endpoints                                  | Backend | Config API complete       |
| Report + Record API endpoints (read-only, paginated)                 | Backend | Report viewing API live   |
| Integration tests: full collect → parse → store pipeline             | QA      | Pipeline verified         |

### Phase 3 — Analysis & Detection (Weeks 7–9)

| Task                                                                | Owner   | Output           |
|---------------------------------------------------------------------|---------|------------------|
| Analysis worker: SPF/DKIM/DMARC alignment checks, conformance rates | Backend | Analysis engine  |
| Correlation worker: anomaly detection, spoofing classification      | Backend | Detection engine |

| Task                                                                         | Owner   | Output              |
|------------------------------------------------------------------------------|---------|---------------------|
| Scoring worker: risk formula, GeolIP + ASN enrichment, AbuseIPDB integration | Backend | Risk scores live    |
| Source entity management + score history                                     | Backend | Source intelligence |
| Analytics API endpoints: conformance, risk trends, top sources               | Backend | Analytics API       |
| Elasticsearch aggregation queries for SOC dashboards                         | Backend | ES queries tuned    |

## Phase 4 — Alerting & SOC Integration (Weeks 10–11)

| Task                                                            | Owner   | Output                 |
|-----------------------------------------------------------------|---------|------------------------|
| Alert service: creation, severity routing, lifecycle management | Backend | Alert system           |
| Alert API: full CRUD + status transitions + annotation          | Backend | Alert API              |
| WebSocket endpoint: real-time alert push                        | Backend | Live dashboard feed    |
| Email + Slack + SIEM notification channels                      | Backend | Notifications live     |
| Recommendation worker: maturity evaluation, auto-generation     | Backend | Recommendations engine |
| IOC feed endpoint: JSON + CSV export                            | Backend | IOC feed live          |

## Phase 5 — Hardening & Deployment (Weeks 12–13)

| Task                                                                 | Owner    | Output               |
|----------------------------------------------------------------------|----------|----------------------|
| K8s manifests + Kustomize overlays (staging + production)            | DevOps   | K8s ready            |
| Prometheus metrics + Grafana dashboards + Loki logging               | DevOps   | Observability stack  |
| OTel tracing instrumentation across all services                     | Backend  | Distributed traces   |
| Security audit: OWASP, bandit, Trivy, RLS test suite                 | Security | Security sign-off    |
| Load testing: k6 on parse pipeline and API (target: 1000 reports/hr) | QA       | Performance baseline |
| Full end-to-end test with real DMARC reports (Google + Microsoft)    | QA       | E2E verified         |
| Documentation: OpenAPI spec finalization, deployment runbook         | All      | Docs complete        |



## 15. Full Python Dependency List

Managed via uv with a locked pyproject.toml. Key packages:

| Package                               | Version | Purpose                 |
|---------------------------------------|---------|-------------------------|
| fastapi                               | ^0.111  | ASGI web framework      |
| uvicorn[standard]                     | ^0.30   | ASGI server             |
| gunicorn                              | ^22     | Process manager         |
| pydantic                              | ^2.7    | Data validation         |
| pydantic-settings                     | ^2.3    | Config from env         |
| sqlalchemy[asyncio]                   | ^2.0    | Async ORM               |
| asyncpg                               | ^0.29   | PostgreSQL async driver |
| alembic                               | ^1.13   | DB migrations           |
| celery[redis]                         | ^5.4    | Task queue              |
| redis                                 | ^5.0    | Redis async client      |
| elasticsearch[async]                  | ^8.13   | ES client               |
| minio                                 | ^7.2    | MinIO/S3 client         |
| aiimaplib                             | ^1.1    | Async IMAP client       |
| defusedxml                            | ^0.7    | Safe XML parsing        |
| dnspython                             | ^2.6    | DNS lookups             |
| maxminddb                             | ^2.5    | GeoIP database reader   |
| pyasn                                 | ^1.6    | ASN lookups             |
| python-jose[cryptography]             | ^3.3    | JWT RS256               |
| passlib[bcrypt]                       | ^1.7    | Password hashing        |
| slowapi                               | ^0.1    | Rate limiting           |
| httpx                                 | ^0.27   | Async HTTP client       |
| aiosmtplib                            | ^3.0    | Async SMTP              |
| cryptography                          | ^42     | Fernet encryption       |
| opentelemetry-sdk                     | ^1.25   | Distributed tracing     |
| opentelemetry-instrumentation-fastapi | ^0.46b0 | FastAPI OTel            |
| opentelemetry-instrumentation-celery  | ^0.46b0 | Celery OTel             |
| prometheus-fastapi-instrumentator     | ^7.0    | Prometheus metrics      |

| Package        | Version | Purpose                   |
|----------------|---------|---------------------------|
| structlog      | ^24.2   | Structured JSON logging   |
| pytest         | ^8.2    | Test runner               |
| pytest-asyncio | ^0.23   | Async test support        |
| pytest-cov     | ^5.0    | Coverage reports          |
| httpx          | ^0.27   | Test client (AsyncClient) |
| ruff           | ^0.5    | Linting + formatting      |
| mypy           | ^1.10   | Static type checking      |
| bandit         | ^1.7    | Security linting          |

## 16. Future Extensions (Bonus Track)

As defined in section 5 of PFE03-2026-SOC, the following enhancements are planned post-MVP:

| Feature                 | Technology                                | Description                                                                                                          |
|-------------------------|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| ML Anomaly Detection    | scikit-learn / PyTorch + Celery ML worker | Unsupervised clustering of source behavior; LSTM for time-series volume anomaly; replaces threshold-based heuristics |
| BIMI & Brand Indicators | DNS lookup + VMC validation API           | Validate BIMI DNS records, check VMC certificate chain, surface BIMI readiness score in dashboard                    |
| Forensic RUF Analysis   | RUF parser module + ES forensic index     | Parse RUF failure reports, extract full email headers and auth chain, correlate with RUA incidents                   |
| Multi-Tenant SaaS Mode  | Stripe API + tenant provisioning worker   | Self-service tenant onboarding, plan-based feature gating, usage metering per tenant                                 |

### End of Backend Deployment Plan

NEXTSTEP — DMARC Analyzer Platform — PFE03-2026-SOC — March 2026